final project

Техническое задание

Финальный проект по Android-разработке

Развитие существующего Android-приложения в персонализированный offline-first mini-product

---

1. Общая постановка задачи

В качестве финальной работы необходимо взять **существующий проект из предыдущих домашних заданий** и развить его до состояния небольшого законченного приложения. Базовый проект должен оставаться узнаваемым и рабочим: список, деталка, навигация, работа с сетью, локальная БД и реактивное обновление UI должны сохраниться.

Финальная работа должна по объёму и сложности соответствовать **2-3 обычным заданиям**. Оцениваться будет не только наличие отдельных технологий, но и то, насколько приложение стало **цельным продуктом**, в котором есть собственная логика, пользовательские данные, офлайн-поведение и законченные сценарии использования.

---

2. Требования к проекту

Финальный проект должен перестать быть просто клиентом открытого API. В нём должна появиться **личная часть приложения**, связанная с действиями и данными конкретного пользователя. Речь не о формальном экране “профиля”, а о собственных пользовательских разделах, собственных данных и собственных сценариях, которые живут не только вокруг ответа сервера. **Внешнее API в финальном проекте должно быть частью системы и источником контента, но не центром приложения: основная ценность работы должна определяться локальной моделью, пользовательскими данными и логикой самого приложения.**

В проекте обязательно должны присутствовать:

- **экран настроек**;
- **экран истории / recent**;

- **как минимум один отдельный пользовательский раздел**, связанный с персональными данными или персонализированным поведением приложения.

Это могут быть личные заметки, собственные списки или коллекции, пользовательские статусы, сохранённые фильтры или пресеты, скрытые элементы, закреплённые элементы, пользовательские теги или иные локальные сущности сопоставимой сложности.

При этом действуют следующие требования:

- пользовательские данные должны **создаваться, изменяться или управляться пользователем**;
- они должны **влиять на поведение приложения**, а не просто храниться формально;
- они не могут сводиться только к одному boolean-флагу или одному toggle;

Если избранное присутствует, оно может быть засчитано только как часть более сложной логики, например при наличии:

- политики актуальности данных;
- TTL / срока жизни кэша;
- фонового обновления;
- предзагрузки для офлайна;
- дополнительных состояний;
- синхронизации;
- отдельных пользовательских сценариев.

Финальный проект должен содержать **минимум 3 новых законченных пользовательских сценария**. Под законченным сценарием понимается не отдельная кнопка, не один экран и не один toggle, а новая функциональность, которая проходит через несколько слоёв приложения и даёт пользователю завершённый результат.

Дополнительно:

- как минимум **2 сценария** должны быть существенными по сложности;
- как минимум **1 сценарий** должен приводить к появлению новой сущности в локальной модели данных или новой связи между сущностями.

Локальное хранение должно быть организовано осмысленно:

- `DataStore` используется для **настроек и предпочтений приложения**;
- `Room` используется для **пользовательских данных, истории, кэша, локальных сущностей и связей между ними**;

- пользовательские данные уровня заметок, списков, истории, состояний, коллекций и аналогичных сущностей **не должны храниться в** `DataStore`.

Проект должен быть построен по принципу **offline-first**. Это означает, что:

- пользователь должен иметь доступ к значимой части данных **без сети**;
- локальные данные должны **реально использоваться UI**;
- сеть должна использоваться для **обновления и синхронизации**, а не как единственный источник состояния экрана;
- если в локальном хранилище уже есть данные, отсутствие интернета не должно делать приложение полностью неработоспособным.

Фоновая обработка через `WorkManager` должна использоваться как часть этой логики и обслуживать работу приложения.

Тесты обязательны. В проекте должны быть тесты на новую ключевую функциональность:

- на бизнес-логику;
- на реактивную или экранную логику;
- на offline / sync / background-часть.

---

3. Допустимое расширение

В качестве одного из крупных сценариев допускается реализация **локальных профилей пользователей** без серверной авторизации. В этом случае приложение должно поддерживать создание и выбор профиля, хранение активного пользователя и раздельные пользовательские данные для разных профилей. Настройки, история, списки, заметки, состояния и иные персональные данные должны быть разделены по пользователям и корректно восстанавливаться после перезапуска приложения.

---

4. Условия приёмки

Работа не принимается, если проект по сути остался тем же приложением со списком и деталкой, а финальное расширение сводится к экрану настроек, теме, простому избранному и одной фоновой задаче.
Не принимается работа, в которой новые функции представляют собой набор несвязанных мелких доработок без законченных сценариев, отсутствуют новые пользовательские данные, `DataStore` используется как замена базы данных, `Room`

используется только формально и не влияет на поведение приложения, offline-first заявлен только на словах, а без сети приложение фактически бесполезно при наличии локальных данных. Также не принимается работа без тестов на новую существенную функциональность.

---

5. Что сдавать

- **PR** с рабочим проектом в котором все файлы задания;
- В PR не должно быть файлов, добавленных .gitignore
- **README**, в котором кратко указано:
  - что именно было добавлено в финальной работе;
  - какие новые пользовательские данные появились;
  - какие новые сценарии реализованы;
  - как устроена offline-first часть;
  - где и зачем используется фоновая обработка;
  - какие тесты написаны.

При необходимости могут быть приложены дополнительные материалы для демонстрации работы проекта.

---

6. Критерии оценки

Оцениваться будут работоспособность и целостность проекта, глубина расширения функциональности, качество локальной модели данных, качество offline-first реализации, осмысленность фоновой обработки, тестирование, архитектура решения и качество реализации в целом.

1. Работоспособность и выполнение обязательных требований (0–4)
   - **0** — проект не собирается, не запускается или критически не соответствует ТЗ
   - **2** — проект запускается, но часть обязательных требований не выполнена или есть серьёзные проблемы в ключевых сценариях
   - **4** — проект рабочий, обязательные требования выполнены, ключевые сценарии реализованы, критичных проблем нет
2. Глубина расширения проекта и качество новых сценариев (0–3)
   - **0** — расширение слабое, новые сценарии почти отсутствуют
   - **1** — новые функции есть, но они поверхностные или фрагментарные
   - **2** — реализовано несколько законченных сценариев, но глубина неравномерная

- **3** — проект существенно расширен: есть новые сущности, законченные сценарии и ощутимая новая ценность для пользователя

3. Качество технической реализации (0–2)
   - **0** — реализация формальная или слабая
   - **1** — техническая база есть, но заметная часть решений поверхностна или не доведена
   - **2** — реализация цельная и осмысленная: технические решения поддерживают продуктовую логику, тесты покрывают важную новую функциональность
4. Субъективный балл (-1..1)

Дедлайн: до **01.06.2026** - без штрафов. После снимается **1 балл**, далее **минус 1 балл за каждую неделю просрочки**, максимум до **−4 баллов**

---

7. Примеры сильного уровня сложности

Сильной финальной работой считается проект, в котором внешний контент из API встроен в собственную локальную систему приложения: например, есть несколько типов пользовательских сущностей, раздельное хранение пользовательских данных, собственные списки или коллекции, заметки, статусы, сохранённые фильтры, история, персонализированные разделы, политика актуальности данных, офлайн-предзагрузка, фоновая актуализация и тестируемая логика синхронизации.

Ещё один пример - приложение, в котором пользовательские данные образуют самостоятельный слой поверх API: со своими сущностями и связями, правилами хранения и устаревания данных, локальными профилями пользователей, разделением данных между ними, восстановлением состояния после перезапуска и полноценной работой ключевых сценариев без сети.